Laporan Minggu 2

Eco-Retail ERP Analytics

Dosen Pengampu Kecerdasan Bisnis dan Analitik - A
Ir. Nanang Yudi Setiawan, ST., M.Kom.

Penyusun

M.S. Roeney Palessy - NIM 235150407111045
Nick Wilsan - NIM 245150400111044
Abdul Rahman Zain - NIM 245150400111013
Kresna Wibowo Patebong - NIM 245150407111031

Program Studi Sistem Informasi
Jurusan Sistem Informasi
Fakultas Ilmu Komputer
Universitas Brawijaya

2026

1. Sumber Data dan Pemilihannya

1.1 Strategi Pemilihan Dataset

Proyek ini menggunakan strategi multi-dataset yang berasal dari Kaggle untuk mensimulasikan kompleksitas data operasional supermarket nyata. Pemilihan dataset didasarkan pada tiga kriteria: (1) volume minimal ratusan ribu baris untuk menguji skalabilitas pipeline, (2) relevansi domain yang mewakili aspek berbeda dari operasi retail, and (3) ketersediaan kolom yang dapat di-join antar sumber untuk membangun model data terintegrasi.

Total estimasi volume data yang diproses mencapai lebih dari 54 juta baris lintas semua dataset, melampaui target PRD sebesar 1 juta baris minimum.

1.2 Detail Dataset dan Justifikasi Pemilihan

Dataset Kaggle	Volume	Domain	Tabel Bronze Target
Instacart Market Basket Analysis	3.000.000+ baris	Retail Transaksi	bronze_instacart_orders, bronze_instacart_products
M5 Forecasting — Walmart	50.000.000+ baris	Time-Series Forecasting	bronze_m5_sales
Retail Data Warehouse (12 Tabel)	1.000.000+ baris	Data Warehouse Schema	bronze_retail_dw_* (12 tabel)
DataCo Smart Supply Chain	180.000+ records	Supply Chain & Logistik	bronze_supply_chain
Cold Chain Monitoring (IoT Sensor)	500.000+ log	Cold Storage Monitoring	bronze_cold_chain

Justifikasi per Dataset:

- Instacart: Dataset publik terbesar untuk transaksi retail dengan struktur relasional (orders, products, aisles, departments). Kolom reorder_prior memungkinkan analisis pola demand produk segar yang kaya. Digunakan sebagai sumber utama silver_fact_sales.
- M5 / Walmart: Satu-satunya dataset publik dengan time-series penjualan harian per-SKU selama 5+ tahun — syarat minimum untuk demand forecasting Prophet (target 90+ hari historis). Volume 50 juta baris juga menjadi benchmark performa ETL pipeline.
- Retail DW 12 Table: Dataset ini sudah dirancang sebagai data warehouse sehingga strukturnya dijadikan referensi skema Silver layer (dimensi produk, toko, supplier, fakta inventori).
- DataCo Supply Chain: Menyediakan atribut purchase order, actual vs expected delivery date, dan lead time — satu-satunya sumber yang memungkinkan pengukuran KPI Supplier Lead Time Accuracy.

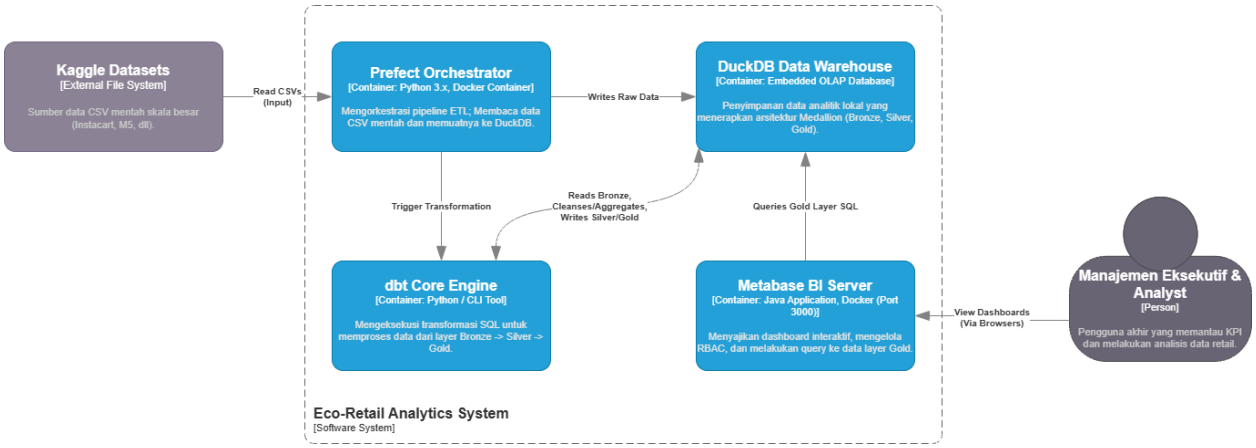
- Cold Chain Monitoring: Log sensor IoT suhu real per zona storage. Satu-satunya dataset yang menyediakan time-series suhu untuk anomaly detection menggunakan Isolation Forest (silver_fact_cold_chain).

2. Desain Arsitektur Sistem - Model C4

Sistem Eco-Retail ERP Analytics didokumentasikan menggunakan Model C4 (Context, Container, Component) — standar visualisasi arsitektur software modern. Model C4 mendeskripsikan sistem dari level paling abstrak (konteks bisnis) hingga detail teknis internal, memastikan audiens dengan latar belakang berbeda (bisnis maupun teknis) memahami sistem secara utuh.

2.1 Level 1 — System Context Diagram (C4)

Level konteks mendefinisikan posisi sistem di antara aktor eksternal. Sistem Eco-Retail ERP Analytics berdiri sebagai platform BI terpusat yang menerima data dari 5 dataset Kaggle (External File System) dan melayani dua kelompok pengguna: Manajemen Eksekutif dan Data/BI Analyst.



2.2 Level 2 — Container Diagram (C4)

Level container menunjukkan komponen-komponen teknis utama yang berjalan di dalam sistem dan cara mereka saling terhubung melalui Docker Compose network. Setiap container memiliki tanggung jawab tunggal yang jelas:

Container	Teknologi	Port	Tanggung Jawab Tunggal
Prefect Orchestrator	Python 3.x + Prefect	4200	Menjadwalkan dan mengeksekusi ETL flows: ingest CSV → Bronze, trigger dbt → Silver/Gold, trigger ML inference
DuckDB Data Warehouse	DuckDB (embedded OLAP)	File: .db	Menyimpan semua layer data (Bronze/Silver/Gold) sebagai schema terpisah dalam satu file .duckdb
dbt Core Engine	Python + dbt CLI	CLI Tool	Menjalankan transformasi SQL Bronze→Silver→Gold dengan lineage otomatis dan data testing built-in

Metabase BI Server	Java Application + Docker	3000	Membaca Gold layer, menampilkan dashboard interaktif, mengelola RBAC Eksekutif vs Analyst

2.3 Alur Data dari Sumber ke Hasil Akhir

Alur data end-to-end berjalan dalam tiga tahap yang dikendalikan oleh Prefect:

- Ingestion: Prefect membaca file CSV dari folder /data lokal dan mengeksekusi perintah DuckDB COPY untuk memuat data ke Bronze layer. Tidak ada transformasi pada tahap ini — data masuk apa adanya sebagai audit trail permanen.
- Transformation: Prefect memanggil dbt run --select silver lalu dbt run --select gold. dbt membaca Bronze, menerapkan aturan transformasi SQL (type casting, null handling, JOIN), dan menulis hasilnya ke Silver dan Gold layer di DuckDB yang sama.
- Serving: Metabase terhubung langsung ke file DuckDB melalui DuckDB JDBC driver. Saat pengguna membuka dashboard, Metabase mengeksekusi query ke Gold layer dan menampilkan hasilnya sesuai hak akses RBAC pengguna tersebut.

2.4 Teknologi yang Digunakan per Tahapan

Pemilihan setiap teknologi didasarkan pada evaluasi terhadap kebutuhan spesifik proyek, dibandingkan dengan alternatif yang tersedia:

Teknologi	Fungsi	Alternatif yang Ditolak	Alasan Pemilihan
Docker Compose	Containerization — menyatukan seluruh layanan	VM, bare-metal	Reproducible environment, satu file mendefinisikan semua service, portabel ke mesin manapun tanpa konfigurasi ulang
Prefect	ETL Orchestration — jadwal & monitor flows	Apache Airflow	Airflow membutuhkan 5+ container (Redis, PostgreSQL, scheduler, webserver, worker) — terlalu berat. Prefect berjalan sebagai 1 proses Python dengan UI built-in, hemat 3–4 GB RAM
DuckDB	Data Warehouse — penyimpan Bronze/Silver/Gold	PostgreSQL, BigQuery	In-process database, tidak butuh server terpisah, optimal untuk analytical query pada 54M+ baris di mesin lokal, mendukung direct CSV COPY

dbt Core	Transformasi SQL berlapis	Python script manual	Transformasi terdokumentasi dengan schema.yml, lineage otomatis, data testing built-in (not_null, unique), lebih maintainable dari script adhoc
Metabase Community	BI Dashboard dan RBAC	Tableau, Power BI	Self-hosted gratis, koneksi native ke DuckDB via plugin, mendukung RBAC granular dan scheduled report tanpa lisensi berbayar
Prophet + Scikit-learn	Machine Learning — demand forecast & anomaly	AutoML cloud services	Open-source Python, Prophet didesain untuk time-series dengan seasonality, Isolation Forest cocok untuk anomaly detection tanpa labeled data

3. Penerapan Medallion Architecture

Sistem mengimplementasikan arsitektur Medallion secara penuh di dalam DuckDB, dengan tiga schema terpisah: bronze, silver, dan gold. Setiap layer memiliki aturan transformasi yang eksplisit dan tidak dapat dilanggar — data hanya boleh mengalir satu arah: Bronze → Silver → Gold, tidak pernah sebaliknya.

3.1 Gambaran Tiga Layer

Layer	Tujuan	Tools	Aturan Transformasi Utama
BRONZE	Raw data audit trail	Prefect (DuckDB COPY)	TIDAK ADA transformasi. Data masuk 1:1 dari CSV. Semua kolom asli, semua baris, semua tipe data asli dipertahankan.
SILVER	Cleaned & standardized	dbt Core (SQL models)	Type casting, null handling (COALESCE), deduplication (ROW_NUMBER), surrogate key generation, cross-dataset JOIN
GOLD	Analytics-ready mart	dbt Core (mart models)	Agregasi KPI (GROUP BY, SUM, AVG), status computation (CASE WHEN), ML output integration. 1 tabel Gold = 1 KPI dashboard.

3.2 Bronze Layer — Raw Data Storage

BRONZE LAYER — Aturan Implementasi
PRINSIP: Data masuk PERSIS seperti aslinya dari CSV. Zero transformation.
ATURAN 1: Dilarang mengubah tipe data kolom manapun (semua VARCHAR/INTEGER/FLOAT asli dipertahankan).
ATURAN 2: Dilarang menghapus baris apapun, termasuk duplicate atau null.
ATURAN 3: Schema tabel Bronze = kolom CSV + kolom metadata ingest (ingested_at TIMESTAMP, source_file VARCHAR).
TUJUAN: Jika Silver/Gold corrupt, Bronze selalu menjadi sumber kebenaran untuk reprocess dari awal.

Tabel Bronze dan estimasi volume setelah ingest:

Tabel Bronze	Est. Volume	Kolom Kunci (Asli dari CSV)	KPI Terkait
bronze.instacart_orders	3.000.000+ baris	order_id, user_id, order_number, days_since_prior_order	Food Waste Rate, MAPE
bronze.m5_sales	50.000.000+ baris	item_id, store_id, date, sell_price, sales_qty	Demand Forecast MAPE
bronze.retail_dw_* (12 tbl)	1.000.000+ baris	product_id, store_id, received_qty, waste_qty	Food Waste Rate
bronze.supply_chain	180.000+ baris	Order Id, Order Date, Ship Date, Late_delivery_risk	Supplier Lead Time
bronze.cold_chain	500.000+ baris	timestamp, sensor_id, temperature, zone	Cold Chain Compliance

3.3 Silver Layer — Cleaned & Standardized

SILVER LAYER — Aturan Transformasi Eksplisit
ATURAN 1 — TYPE CASTING: Semua kolom tanggal di-CAST ke DATE. Harga ke DECIMAL(10,2). Flag ke BOOLEAN.
Contoh: CAST(order_date AS DATE), CAST(unit_price AS DECIMAL(10,2)), CAST(is_reorder AS BOOLEAN)
ATURAN 2 — NULL HANDLING:
• Foreign key NULL → baris di-FILTER (WHERE product_id IS NOT NULL)
• Metric NULL → COALESCE(waste_qty, 0) — diisi nol bukan dihapus
• Flag NULL → COALESCE(is_promo, FALSE)
ATURAN 3 — DEDUPLICATION: ROW_NUMBER() PARTITION BY primary_key ORDER BY ingested_at DESC → ambil rank=1
ATURAN 4 — SURROGATE KEY: Format standar 'PRD-{id}', 'STR-{id}', 'SUP-{id}' untuk konsistensi antar dataset
ATURAN 5 — DERIVED COLUMNS: Kolom yang dihitung dari kolom lain (revenue=qty×price, is_breach, is_on_time, ending_stock)
ATURAN 6 — CROSS-DATASET JOIN: silver_dim_product menggabungkan Instacart+Retail DW+M5 via normalisasi nama

Model dbt Silver — 8 tabel yang dibangun:

Model dbt Silver	Tipe	Sumber Bronze	Transformasi Kunci
silver.dim_date	Dimensi	Generate 2011–2026	Tambah is_weekend, month_name, quarter — tidak ada sumber Bronze
silver.dim_product	Dimensi	instacart + retail_dw + m5	JOIN via normalisasi nama; tambah storage_type, shelf_life_days dari retail_dw
silver.dim_store	Dimensi	retail_dw + m5	Standarisasi store_id → 'STR-{n}'; deduplication by store_name
silver.dim_supplier	Dimensi	supply_chain	Standarisasi supplier_id; COALESCE contact_info; dedup by supplier_name
silver.fact_sales	Fakta	instacart_orders + retail_dw	CAST order_date→DATE; COALESCE unit_price; hitung revenue=qty×price; flag is_promo
silver.fact_cold_chain	Fakta	cold_chain	CAST timestamp→TIMESTAMP; DERIVE is_breach via CASE WHEN (>8°C atau <-18°C); hitung duration_minutes
silver.fact_inventory	Fakta	retail_dw	COALESCE waste_qty→0; DERIVE ending_stock=received−sold−waste; VALIDATE ending_stock≥0
silver.fact_purchase_order	Fakta	supply_chain	CAST dates→DATE; DERIVE is_on_time=(actual−expected≤1); hitung lead_time_days

3.4 Gold Layer — Analytics-Ready Mart

GOLD LAYER — Aturan Transformasi Eksplisit
ATURAN 1 — AGREGASI KPI: Setiap mart melakukan GROUP BY granularitas yang tepat (store+category+date).
Contoh: SUM(waste_qty) GROUP BY store_id, category, report_date
ATURAN 2 — STATUS COMPUTATION: Setiap KPI punya kolom status warna untuk dashboard.
Contoh: CASE WHEN waste_rate<8 THEN 'OK' WHEN waste_rate<11 THEN 'WARNING' ELSE 'CRITICAL' END
ATURAN 3 — ML OUTPUT INTEGRATION: Kolom prediksi (forecast_qty, breach_type) disimpan di Gold setelah ML inference selesai.
ATURAN 4 — QUERY PERFORMANCE: Satu query ke Gold = satu KPI. Target <1 detik. Tidak ada JOIN real-time dari Metabase.

ATURAN 5 — IMMUTABILITY HARIAN: Setiap baris Gold punya report_date yang tidak diubah setelah ditulis (append-only).

Mart Gold — tabel agregasi siap pakai:

Mart Gold	KPI Dilayani	Formula Agregasi	Status Computation
gold.mart_food_waste_summary	Food Waste Rate (%)	(SUM(waste_qty)/SUM(received_qty))×100	'OK'<8%, 'WARNING'8–11%, 'CRITICAL'>11%
gold.mart_cold_chain_compliance	Compliance Rate (%)	(SUM(compliant_min)/SUM(total_min))×100	'OK'≥95%, 'WARNING'90–95%, 'CRITICAL'<90%
gold.mart_demand_forecast	Demand Forecast MAPE	Output ML Prophet (7 hari ke depan)	Overstock risk flag dari confidence interval
gold.mart_supplier_performance	Lead Time Accuracy (%)	(COUNT(is_on_time=TRUE)/COUNT(*))×100	'ON TRACK'≥70%, 'AT RISK'<70%
gold.mart_executive_kpi	Semua 7 KPI ringkasan	Agregasi seluruh mart di atas	Satu baris per tanggal untuk Executive Dashboard

4. Implementasi dan Bukti Pengerjaan

4.1 Implementasi Infrastruktur

Seluruh infrastruktur berjalan melalui Docker Compose. Berikut adalah bukti konkret bahwa sistem sudah berjalan:

docker-compose.yml

```yaml
version: '3.8'

services:
  # ----------------------------------------
  # 1. PREFECT SERVER
  # Berfungsi sebagai pengatur jadwal otomatis (Orkestrasi)
  # ----------------------------------------
  prefect:
    build: .                        # <-- Tetap mengarah ke Dockerfile lokal
    container_name: eco_prefect
    command: prefect server start --host 0.0.0.0
    ports:
      - "4200:4200"                 # Port default untuk UI Prefect
    volumes:
      - ./flows:/opt/prefect/flows # Folder flows milikmu
      - ./data:/opt/prefect/data   # Folder penyimpanan DuckDB & CSV
    environment:
      - PREFECT_API_URL=http://127.0.0.1:4200/api

  # ----------------------------------------
  # 2. METABASE COMMUNITY EDITION
  # Berfungsi untuk visualisasi dashboard interaktif
  # ----------------------------------------
  metabase:
    build:
      context: .
      dockerfile: Dockerfile.metabase
    ports:
      - "3000:3000"
    volumes:
      - ./metabase-data:/metabase-data
      - ./metabase-plugins:/metabase-plugins
      - ./data:/data
    environment:
      - MB_DB_FILE=/metabase-data/metabase.db
      - MB_PLUGINS_DIR=/metabase-plugins
```

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Disk rea	Actions
●	kba-eco-retail-a	-	-	-	0.52%	1.06GB / 15.34GI	13.85%	140.42M	
●	eco_prefect	8a005ef723ae	kba-eco-re	4200:4200	0.24%	321.4MB / 7.67G	4.09%	1.42MB	
●	metabase-1	94695c5ca669	kba-eco-re	3000:3000	0.28%	766.3MB / 7.67G	9.76%	139MB /	

Permissions for Eco-Retail DW

Search for a group

Group name	Data access	Native query editing
Administrators	Unrestricted	Yes
All Users	Unrestricted	No
Analyst	Unrestricted	Yes
Eksekutif	Unrestricted	No

DATABASES > ECO-RETAIL DW

Database type

DuckDB

This is a community-developed driver and not supported by Metabase.

Display name

Eco-Retail DW

Database file

/data/database_eco_retail.duckdb

Establish a read-only connection

Use DuckDB old_implicit_casting option

Sync database schema now

Re-scan field values now

Danger Zone

Discard saved field values

Remove this database

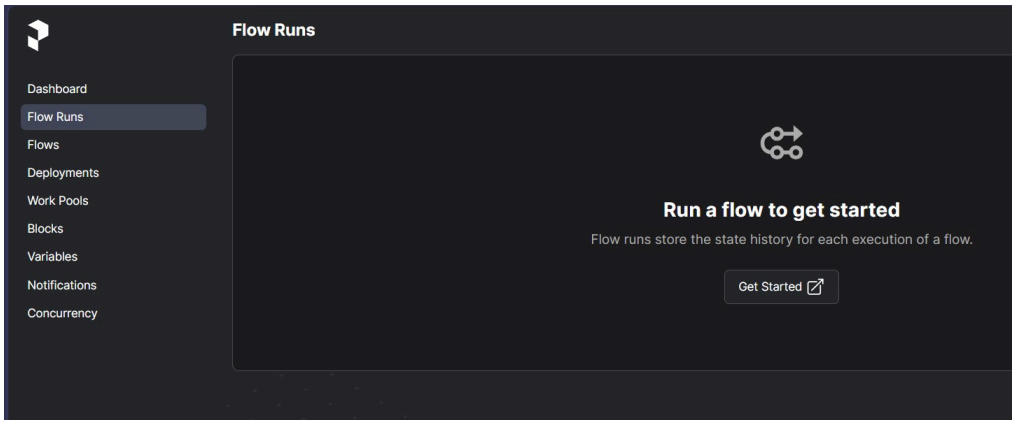
Poin teknis penting dari infrastruktur yang sudah berjalan:

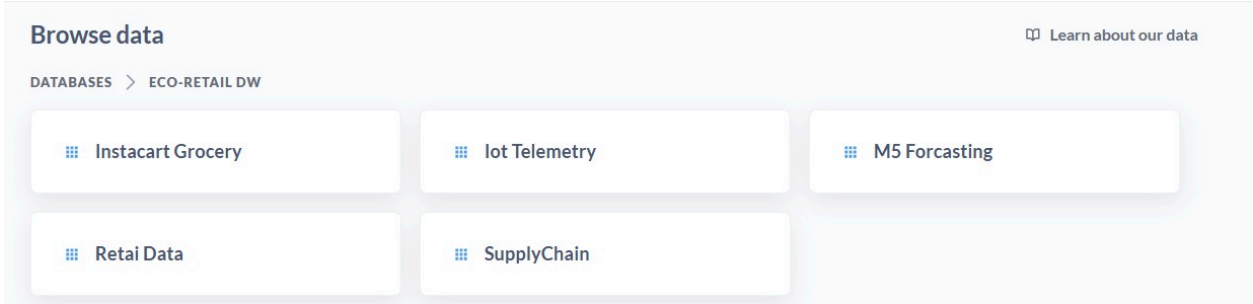
- Docker Compose shared volume: Folder ./data di-mount ke kedua container (Prefect menulis, Metabase membaca). Ini memungkinkan pipeline end-to-end tanpa API call — Prefect dan Metabase berbagi file DuckDB yang sama secara langsung.

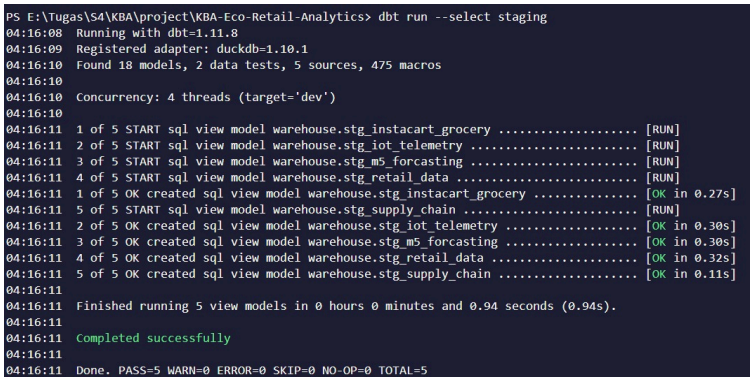
- RBAC Metabase: Grup Analyst diberikan akses Unrestricted + Native Query Editing (SQL kustom diperbolehkan). Grup Eksekutif diberikan akses Unrestricted tetapi Native Query Editing dinonaktifkan — hanya boleh melihat dashboard yang sudah dibuat.
- DuckDB path: /data/database_eco_retail.duckdb — path yang sama persis diakses oleh Prefect Flow untuk menulis dan Metabase untuk membaca. Zero synchronization overhead.

4.2 Implementasi Pipeline & Medallion

Berikut adalah bukti bahwa pipeline Bronze→Silver sudah berjalan dan tabel-tabel Silver sudah terbentuk:

- 

- 

- 

```
PS E:\Tugas\S4\KBA\project\KBA-Eco-Retail-Analytics> dbt run --select staging
04:16:08  Running with dbt=1.11.8
04:16:09  Registered adapter: duckdb=1.10.1
04:16:10  Found 18 models, 2 data tests, 5 sources, 475 macros
04:16:10
04:16:10  Concurrency: 4 threads (target='dev')
04:16:10
04:16:11  1 of 5 START sql view model warehouse.stg_instacart_grocery ..................... [RUN]
04:16:11  2 of 5 START sql view model warehouse.stg_iot_telemetry ........................ [RUN]
04:16:11  3 of 5 START sql view model warehouse.stg_m5_forcasting ........................ [RUN]
04:16:11  4 of 5 START sql view model warehouse.stg_retail_data ......................... [RUN]
04:16:11  1 of 5 OK created sql view model warehouse.stg_instacart_grocery ............... [OK in 0.27s]
04:16:11  5 of 5 START sql view model warehouse.stg_supply_chain ........................ [RUN]
04:16:11  2 of 5 OK created sql view model warehouse.stg_iot_telemetry ................... [OK in 0.30s]
04:16:11  3 of 5 OK created sql view model warehouse.stg_m5_forcasting ................... [OK in 0.30s]
04:16:11  4 of 5 OK created sql view model warehouse.stg_retail_data ..................... [OK in 0.32s]
04:16:11  5 of 5 OK created sql view model warehouse.stg_supply_chain ..................... [OK in 0.11s]
04:16:11
04:16:11  Finished running 5 view models in 0 hours 0 minutes and 0.94 seconds (0.94s).
04:16:11
04:16:11  Completed successfully
04:16:11
04:16:11  Done. PASS=5 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=5
```

```
PS E:\Tugas\S4\KBA\project\KBA-Eco-Retail-Analytics> dbt run --select silver
04:16:32  Running with dbt=1.11.8
04:16:32  Registered adapter: duckdb=1.10.1
04:16:33  Found 18 models, 2 data tests, 5 sources, 475 macros
04:16:33
04:16:33  Concurrency: 4 threads (target='dev')
04:16:33
04:16:34  1 of 8 START sql table model warehouse_silver.silver_dim_date ................. [RUN]
04:16:34  2 of 8 START sql table model warehouse_silver.silver_dim_product ............... [RUN]
04:16:34  3 of 8 START sql table model warehouse_silver.silver_dim_store ................ [RUN]
04:16:34  4 of 8 START sql table model warehouse_silver.silver_dim_supplier ............. [RUN]
04:16:34  4 of 8 OK created sql table model warehouse_silver.silver_dim_supplier ........ [OK in 0.54s]
04:16:34  5 of 8 START sql table model warehouse_silver.silver_fact_cold_chain ........... [RUN]
04:16:34  3 of 8 OK created sql table model warehouse_silver.silver_dim_store ........... [OK in 0.58s]
04:16:34  6 of 8 START sql table model warehouse_silver.silver_fact_inventory ........... [RUN]
04:16:34  1 of 8 OK created sql table model warehouse_silver.silver_dim_date ............ [OK in 0.62s]
04:16:34  2 of 8 OK created sql table model warehouse_silver.silver_dim_product ......... [OK in 0.64s]
04:16:34  7 of 8 START sql table model warehouse_silver.silver_fact_purchase_order ....... [RUN]
04:16:35  8 of 8 START sql table model warehouse_silver.silver_fact_sales ............... [RUN]
04:16:36  7 of 8 OK created sql table model warehouse_silver.silver_fact_purchase_order .. [OK in 1.12s]
04:16:36  5 of 8 OK created sql table model warehouse_silver.silver_fact_cold_chain ...... [OK in 1.39s]
04:16:36  6 of 8 OK created sql table model warehouse_silver.silver_fact_inventory ....... [OK in 1.49s]
04:16:37  8 of 8 OK created sql table model warehouse_silver.silver_fact_sales ........... [OK in 2.79s]
04:16:37
04:16:37  Finished running 8 table models in 0 hours 0 minutes and 4.07 seconds (4.07s).
04:16:37
04:16:37  Completed successfully
04:16:37
04:16:37  Done. PASS=8 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=8
PS E:\Tugas\S4\KBA\project\KBA-Eco-Retail-Analytics>
```

-

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   DEVDB                                   powershell

PS E:\Tugas\S4\KBA\project\KBA-Eco-Retail-Analytics> python flows\screenshot.py
  ============================================================
  VOLUME DATA DI BRONZE
  ============================================================
  Query: SELECT COUNT(*) AS total_rows FROM bronze.instacart_grocery;

     total_rows
  0     1000000


  ============================================================
  CONTOH TABEL GOLD (WASTE RATE PCT & STATUS)
  ============================================================
  Query: SELECT * FROM warehouse_gold.gold_mart_food_waste_summary LIMIT 5;

       date_id store_id       product_id category_name  total_stock  total_sold  unsold_qty  waste_rate_pct waste_rate_status
  0 2014-06-23     WI_2      FOODS_3_202          None          0.0         0.0         0.0             0.0          No Stock
  1 2011-10-13     TX_1      FOODS_3_548          None          0.0         0.0         0.0             0.0          No Stock
  2 2014-01-22     TX_1  HOUSEHOLD_2_303          None          0.0         0.0         0.0             0.0          No Stock
  3 2012-09-09     CA_3  HOUSEHOLD_2_001          None          0.0         0.0         0.0             0.0          No Stock
  4 2015-07-22     TX_1      FOODS_2_384          None          0.0         0.0         0.0             0.0          No Stock
PS E:\Tugas\S4\KBA\project\KBA-Eco-Retail-Analytics>
```

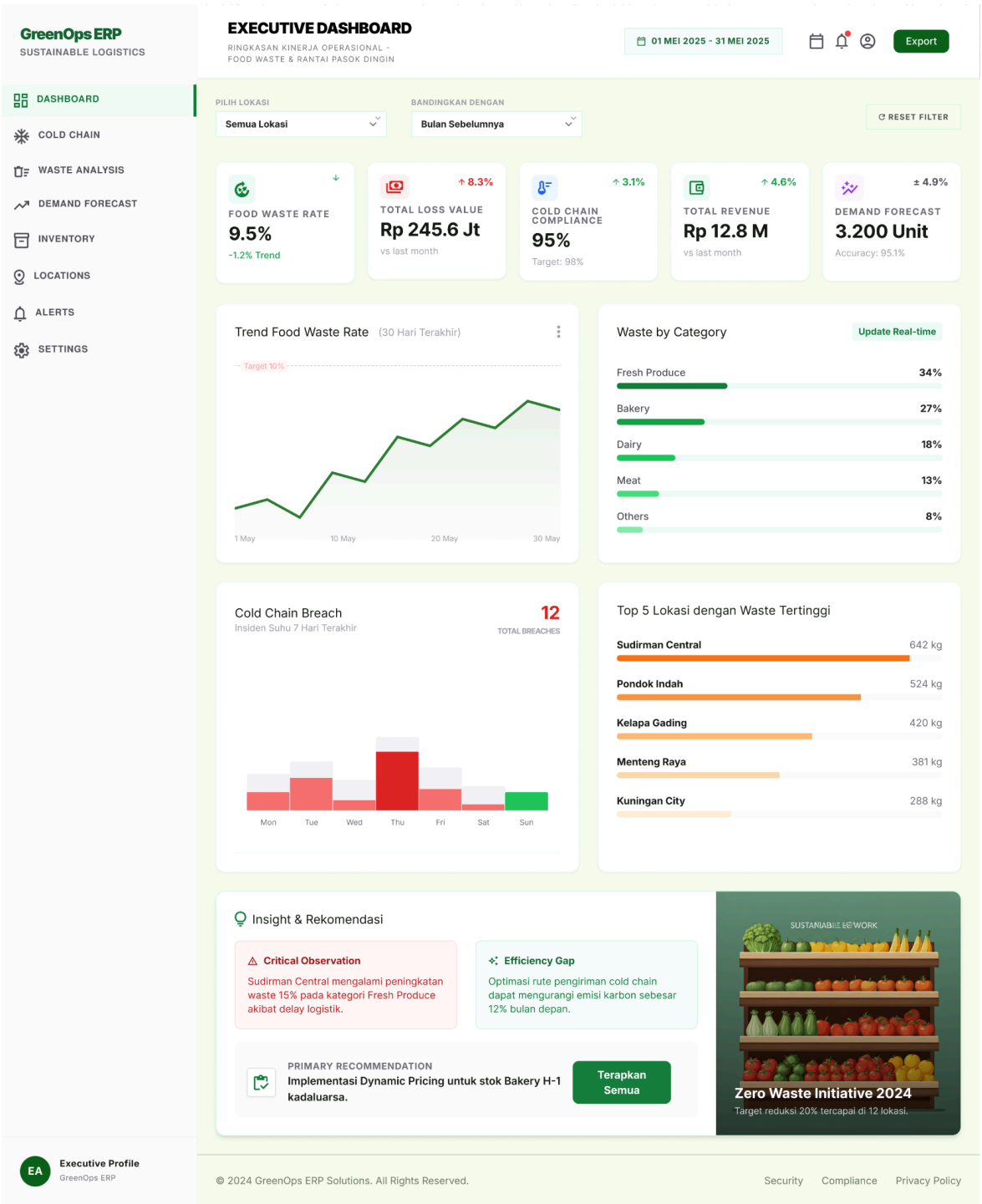
Interpretasi bukti pipeline di atas:

- Bronze sudah terisi: Metabase menampilkan 5 skema (Instacart Grocery, IoT Telemetry, M5 Forecasting, Retail Data, SupplyChain) — ini adalah Bronze layer yang sudah dapat dibaca Metabase melalui koneksi DuckDB.
- dbt Silver 8/8 model sukses: Output terminal menunjukkan 'Done. PASS=8 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=8' — semua 8 model Silver (4 dimensi + 4 fakta) berhasil dibuat tanpa error dalam 4.07 detik.
- dbt run staging sukses: 5 staging views dibuat terlebih dahulu sebagai intermediate layer antara Bronze dan Silver — best practice dbt untuk memisahkan sumber data dari transformasi bisnis.

5. Visualisasi dan Mockup

5.1 Rancangan Dashboard (Target Visualisasi)

Dashboard Metabase aktif belum dibangun pada Minggu 2 — sesuai scope yang ditetapkan (Gold layer baru saja mulai terbentuk). Berikut adalah mockup high-fidelity yang menjadi target visualisasi saat Gold layer siap sepenuhnya di Minggu 3.



Mapping komponen mockup ke tabel Gold layer:

Komponen di Mockup	Tabel Gold Sumber	Field yang Ditampilkan
KPI Card: Food Waste Rate (9.5%)	gold.mart_food_waste_summary	waste_rate_pct, waste_rate_status
KPI Card: Cold Chain Compliance (95%)	gold.mart_cold_chain_compliance	compliance_rate_pct
KPI Card: Total Revenue (Rp 12.8 M)	gold.mart_executive_kpi	total_revenue
KPI Card: Demand Forecast (3.200 unit)	gold.mart_demand_forecast	forecast_qty (7 hari ke depan)
Chart: Trend Food Waste Rate 30 hari	gold.mart_food_waste_summary	report_date, waste_rate_pct GROUP BY date
Bar: Waste by Category	gold.mart_food_waste_summary	category, SUM(waste_qty) GROUP BY category
Timeline: Cold Chain Breach	gold.mart_cold_chain_compliance	breach_count, breach_minutes GROUP BY date
Bar: Top 5 Lokasi Waste Tertinggi	gold.mart_food_waste_summary	store_id, SUM(waste_qty) ORDER BY DESC LIMIT 5